

MODULE 07 · DOMAIN 3 - APPLICATIONS OF FOUNDATION MODELS

Agents & Tool Use

When answering a question is not enough, and something has to actually happen.

DOMAIN 3 of 5 EXAM WEIGHT 28% MODULE 7 of 14

An agent is a model that can **choose what to do next**, call something, look at the result, and decide again.

Reason, act, observe, repeat

01

Reason

Work out what the request needs and which tool would help.

02

Act

Invoke that tool with parameters the model has chosen.

03

Observe

Read what came back, including errors.

04

Repeat

Decide whether the task is done or another step is needed.

05

Respond

Give the user a final answer once the goal is met.

ON THE EXAM

This is often called the ReAct pattern — reasoning and acting interleaved. Each pass through the loop is a fresh model call.

What a Bedrock agent is made of

COMPONENT	WHAT IT DOES
Foundation model	Does the reasoning and picks the next step.
Instructions	A system prompt describing the agent's role and boundaries.
Action groups	The tools it may call, defined by an API schema plus a Lambda function.
Knowledge bases	Documents it can retrieve from, so an agent can do RAG too.
Orchestration	The managed loop that runs reason, act, observe.
Trace	A step-by-step record of what it decided and why.

ON THE EXAM

Trace is the answer for debugging and auditability. It shows the reasoning, the tool calls, and the results at each step.

How an agent is given a tool

- ◆ An **API schema** describes each operation, its parameters, and what it returns.
- ◆ The model reads those descriptions to decide **when** a tool applies. Wording matters.
- ◆ A **Lambda function** receives the chosen parameters and does the real work.
- ◆ Nothing happens outside the tools you define. The agent cannot invent capabilities.
- ◆ Vague descriptions cause wrong tool choices more often than model weakness does.

ON THE EXAM

Tool descriptions are prompt engineering. An agent picking the wrong tool is usually a description problem, not a model problem.

Memory and session state

- ◆ **Session state** carries context across turns within one conversation.
- ◆ **Memory** persists across sessions, so the agent recalls earlier conversations.
- ◆ Short-term memory is the current session. Long-term memory spans sessions.
- ◆ Without either, every request starts from nothing. The model itself remembers nothing.

ON THE EXAM

A scenario about an assistant forgetting a customer between visits is a memory question, not a model capability question.

Return of control

- ◆ The agent can hand a decision back rather than executing it itself.
- ◆ Your application performs the action, then returns the result to the agent.
- ◆ Used when an action needs approval, or must run inside your own systems.
- ◆ The natural place to put a human in the loop before anything irreversible.

ON THE EXAM

When a scenario requires approval before an agent acts, return of control is the mechanism that provides it.

Multi-agent collaboration

- ◆ A **supervisor agent** breaks a request into sub-tasks and delegates them.
- ◆ **Collaborator agents** each specialise in one domain and report back.
- ◆ The supervisor consolidates their outputs into one answer.
- ◆ Suits workflows spanning genuinely different specialisms, such as research, analysis and forecasting.
- ◆ More agents means more model calls. Complexity and cost both rise.

ON THE EXAM

Multi-agent is the answer for complex multi-step workflows needing different specialisms. It is the wrong answer for a task one agent handles.

Agents Classic is closed to new customers

- ◆ **Bedrock Agents** became **Agents Classic** and closed to new customers on 30 July 2026.
- ◆ Existing agents keep running. The model catalogue is frozen and no new features are planned.
- ◆ AWS points new builds at **Bedrock AgentCore**: Runtime, Memory, Gateway, Identity, Harness.
- ◆ The exam still tests action groups and knowledge bases. Learn them anyway.

ON THE EXAM

Same situation as Forecast and Fraud Detector in Module 2. Answer from the documented catalogue, not from what you could build today.

Does this even need an agent?

USE AN AGENT

The path is not known in advance

- Steps depend on what earlier steps return
- The request could go several ways
- Natural language in, action out
- New tools should be usable without new code paths

USE ORDINARY AUTOMATION

The path is fixed

- The same steps run every time
- A Lambda function or Step Functions does it
- Deterministic, testable, far cheaper
- No model reasoning is needed at all

THE TIEBREAKER

If you can draw the flowchart in advance, you do not need an agent. Agents earn their cost when the flowchart cannot be drawn.

Why agents surprise people on the bill

- ◆ **Every loop iteration is another model call**, with the whole history resent.
- ◆ A task that looks like one question can bill many times the tokens.
- ◆ Tool results become prompt tokens on the next iteration.
- ◆ Cap the number of steps, keep tool outputs small, use a smaller model where reasoning allows.

ON THE EXAM

Token amplification is the specific term. The conversation may be short; the billed token count is not.

An agent is only as safe as its tools

- ◆ **Least privilege.** The agent's Lambda gets only the permissions that task needs.
- ◆ Prompt injection against an agent becomes **real actions**, not just bad text.
- ◆ **Guardrails** apply to agents too, filtering input and output.
- ◆ **Return of control** for anything irreversible: payments, deletions, external messages.
- ◆ **Trace** gives you the audit record of what was decided and executed.

ON THE EXAM

The stakes change with agents. An injected instruction to a chatbot produces wrong words. An injected instruction to an agent can move money.

Carry these into the exam

- ◆ Agent equals tools plus a loop plus runtime-decided steps.
- ◆ Action group equals API schema plus Lambda. The schema wording drives tool choice.
- ◆ Approval before acting means return of control.
- ◆ Complex workflow spanning specialisms means multi-agent with a supervisor.
- ◆ Fixed, repeatable steps means Lambda or Step Functions, not an agent.
- ◆ Trace is the answer for debugging and auditing what an agent did.

A utility company processes meter readings every night. Each reading is validated, converted into a billing record, and written to a database. The steps never vary.

An architect proposes building this as a Bedrock agent with action groups.

Is this the right choice?

- ☐ A Yes — agents are designed for multi-step workflows
- ☐ B Yes — but only with multi-agent collaboration
- ☐ C No — use Lambda or Step Functions
- ☐ D No — use a knowledge base instead

◆ PAUSE AND DECIDE

Is this the right choice?

- A

 Yes, multi-step
Multi-step is not the test. Unpredictable is the test.
- B

 Yes, multi-agent
More expensive still, for a task with no branching at all.
- C

 No, Lambda or Step Functions
Fixed steps, no reasoning needed. Cheaper and deterministic.
- D

 No, knowledge base
Retrieval over documents. There is no question being asked.

ON THE EXAM

'The steps never vary' is the whole question. A model that reasons is pure overhead when there is nothing to reason about.

A travel company builds an agent that can cancel bookings and issue refunds through internal APIs.

Compliance requires that no refund above five hundred pounds is issued without a human authorising it.

Which mechanism meets the requirement?

- ☐ A Return of control
- ☐ B Bedrock Guardrails
- ☐ C Lower the agent's temperature
- ☐ D Multi-agent collaboration

◆ PAUSE AND DECIDE

Which mechanism meets the requirement?

- A

 Return of control
Hands the action to your application, where approval happens.
- B

 Bedrock Guardrails
Filters content. Does not gate an action on approval.
- C

 Lower the temperature
Makes decisions more consistent. Not a control.
- D

 Multi-agent collaboration
More agents. Still no human involved.

ON THE EXAM

Guardrails is the tempting answer because it sounds like safety. It governs what is said, not what is executed.

A support agent has one action group for looking up orders and another for checking stock. Users report that it often checks stock when they asked about an order.

The agent uses a large, capable foundation model.

What is the most likely cause?

- ☐ A The model is too small for the task
- ☐ B The action group descriptions are ambiguous
- ☐ C Session state is not enabled
- ☐ D The knowledge base needs syncing

◆ PAUSE AND DECIDE

What is the most likely cause?

- A

 Model too small
The scenario says the model is large and capable.
- B

 Ambiguous descriptions
Tool choice is driven by the schema wording the model reads.
- C

 Session state off
Affects memory across turns, not tool selection.
- D

 Knowledge base sync
Concerns retrieval. No retrieval is involved here.

ON THE EXAM

The scenario rules out the model deliberately. When an agent picks the wrong tool, look at how the tools describe themselves.

An internal agent answers HR questions and can also update employee records. Its Lambda function uses an IAM role with broad database write permissions, for convenience.

A security review flags the design before launch.

What is the primary risk?

- ☐ A The agent will be slow to respond
- ☐ B Prompt injection could cause unintended record changes
- ☐ C Token costs will be too high
- ☐ D The knowledge base may return stale answers

◆ PAUSE AND DECIDE

What is the primary risk?

- A

 Slow responses
Permissions breadth does not affect latency.
- B

 Injection causes real changes
The agent can write anything its role permits.
- C

 High token costs
A real agent concern, but unrelated to IAM breadth.
- D

 Stale knowledge base
A retrieval concern. Not what the review flagged.

ON THE EXAM

The fix is least privilege: scope the Lambda's role to the specific records and operations the task needs.